

EcoJudge - README

Eco-Loop Building Agents — Adversarial Setpoint Court

Watch the 3-minute demo video (demo/demo_video.mp4).

What This Is

Buildings waste energy because their HVAC schedules are static — set once, never adapted to what's actually happening in the building or on the grid. This project makes that schedule adaptive by putting two opposing AI agents in charge of it, refereed by deterministic logic:

- Energy Advocate (LLM) argues for setpoints that cut electricity use, especially during high grid-carbon-intensity hours.
- Comfort Advocate (LLM) argues for setpoints that keep occupants thermally comfortable (PMV close to neutral).
- Judge (plain Python, no LLM) picks the winner using the *actual measured* thermal comfort from the last EnergyPlus simulation — not a prediction, not a vote. If comfort is already at risk, comfort wins outright, no negotiation.

The winning setpoints get written back into the live EnergyPlus model, which is re-simulated, and the loop repeats. If a bad patch crashes the simulation, the system feeds the exact error back to an LLM repair agent, which proposes a fix and retries — a real self-correction loop, not just a try/except that gives up. Once a given state has been debated once, a precedent cache lets future iterations skip the LLM entirely and reuse the ruling — cutting API calls without cutting quality.

Result on a real 3-day run (5-zone commercial building, Chicago weather): 10.1% less total energy than the unmodified baseline schedule, with the judge visibly protecting comfort at multiple points in the transcript rather than letting energy savings run unchecked.

How It Works

```
EnergyPlus (physics ground truth)
  → zone temp / humidity / energy use / PMV
    → Energy Advocate (LLM) proposes setpoints ■■
    → Comfort Advocate (LLM) proposes setpoints ■■ → Judge (deterministic)
      → winning setpoints
        → patched into EnergyPlus
        → re-simulate
        → repeat
```

Full architecture, prompt design, and latency-management details are in ARCHITECTURE.md (ARCHITECTURE.md).

Prerequisites

- EnergyPlus 23.2.0 installed (this project pins to that exact IDD version — models/baseline.idf starts with Version,23.2;). Download: <https://energyplus.net/downloads>
- Python 3.10+

- A free Groq API key (<https://console.groq.com>) — or any OpenAI-compatible OSS LLM endpoint

Setup

```
pip install -r requirements.txt
cp .env.example .env
Edit .env:
```

- GROQ_API_KEY — your key
- ENERGYPLUS_EXE / ENERGYPLUS_IDD — point these at your EnergyPlus 23.2.0 install if it's not at C:/EnergyPlusV23-2-0 (the default)

Run the tests

```
python -m pytest -v
```

27 tests, one of which (`test_ep_bridge.py::test_full_pipeline_runs_energypplus_and_extracts_metrics`) runs a real EnergyPlus simulation — this is the one that needs ENERGYPLUS_EXE/ENERGYPLUS_IDD to be correct. `test_orchestrator.py` covers the self-correction (repair-and-retry) loop against a mocked EnergyPlus bridge, both the success and fallback branches.

Run the closed loop

```
python -m src.orchestrator
```

Prints final per-block metrics as JSON. Writes results/transcript.jsonl (every debate/cache-hit/verdict, one line per event) and results/precedent_cache.json.

To see fresh LLM debates instead of cached verdicts from a prior run, delete the cache first:

```
rm results/precedent_cache.json
python -m src.orchestrator
```

Run the baseline comparison + dashboard

```
python dashboard/run_baseline.py
python dashboard/generate_dashboard.py
```

Produces results/dashboard.png (energy + comfort, baseline vs closed-loop) and prints the total % kWh change.

Run the MCP tool server standalone

```
python -m src.mcp_server
```

Starts a stdio MCP server exposing `get_zone_metrics`, `get_simulation_errors`, `get_carbon_signal` — connect any MCP client to inspect the tool-calling contract independent of the orchestrator.

What's already run and committed

results/ in this repo already contains one full real run's output: baseline metrics, closed-loop metrics, dashboard, transcript, cache, and results/idf_snapshots/iter_0.idf through iter_4.idf (the actual runtime-modified building models the deliverables ask for). You don't have to run anything to see the numbers, but rerunning reproduces them (the precedent cache makes reruns deterministic unless you delete it first).